

226. Invert Binary Tree Easy

Acceptance: 79.7% | [LeetCode](#)

Given the `root` of a binary tree, invert the tree, and return *its root*.

Example 1:

```
**Input:** root = [4,2,7,1,3,6,9]
**Output:** [4,7,2,9,6,3,1]
```

Example 2:

```
**Input:** root = [2,1,3]
**Output:** [2,3,1]
```

Example 3:

```
**Input:** root = []
**Output:** []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 100]`.
- `-100 <= Node.val <= 100`

Claude Opus Solution

Invert Binary Tree — Complete Explanation

1. Intuition

The key insight is beautifully simple: **inverting a binary tree means swapping the left and right children of every node**. Think of it like a mirror reflection. If you place a mirror to the right of the tree, every node's left child becomes its right child and vice versa.

The crucial observation is that this is a **recursive structure**: - To invert a tree, swap the root's two children. - But those children are themselves subtrees that also need inverting.

This naturally maps to a **Depth-First Search (DFS)** pattern. Each node only needs to: 1. Invert its left subtree. 2. Invert its right subtree. 3. Swap them.

2. Approach (Pseudocode)

```
function invertTree(root):
    # Base case: an empty tree (or empty subtree) is already inverted
    if root is None:
        return None

    # Recursively invert the left and right subtrees,
    # then SWAP them by assigning the inverted-right to the left
    # and the inverted-left to the right.
    left_inverted = invertTree(root.left)
    right_inverted = invertTree(root.right)

    root.left = right_inverted # right subtree goes to the left
    root.right = left_inverted # left subtree goes to the right

    return root
```

3. Python Solution

```
from typing import Optional

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
```

```

# ---- Base case ----
# If the node is None, there is nothing to invert.
# This also handles the empty-tree case (root = []).
if root is None:
    return None

# ---- Recursive step ----
# Invert the entire left subtree and the entire right subtree first.
# Python evaluates the right-hand side fully BEFORE assignment,
# so we can swap in a single line safely.
#
# root.left receives the inverted RIGHT subtree
# root.right receives the inverted LEFT subtree
root.left, root.right = (
    self.invertTree(root.right), # becomes the new left child
    self.invertTree(root.left),  # becomes the new right child
)

# Return the (now inverted) root so parent calls can wire it up.
return root

```

Why the one-liner swap is safe: In Python, the entire right-hand side of `a, b = x, y` is evaluated into a tuple *before* any assignment happens. So we don't need a temporary variable — there's no risk of overwriting `root.left` before we use it.

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(n)$	We visit every node exactly once , performing $O(1)$ work (a swap) at each. With <code>n</code> nodes, total work is $O(n)$.
Space	$O(h)$	The recursion stack holds at most one node per level of depth, where <code>h</code> is the tree height. Best/balanced case: $O(\log n)$. Worst case (a degenerate "linked-list" tree): $O(n)$.

Given the constraint (≤ 100 nodes), this is extremely efficient.

5. Alternative Approaches

A) Iterative BFS (using a queue)

Process nodes level-by-level, swapping children as you go. Useful if you want to avoid deep recursion.

```

from collections import deque

class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if root is None:
            return None

        queue = deque([root])
        while queue:
            node = queue.popleft()
            # Swap children
            node.left, node.right = node.right, node.left
            # Enqueue non-null children to continue inverting
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        return root

```

- **Time:** $O(n)$ — **Space:** $O(w)$ where `w` is the max tree width (up to $O(n)$ for the bottom level).

B) Iterative DFS (using a stack)

Identical logic to BFS but with a `stack` instead of a `queue`. The traversal order differs, but the result is the same since every node still gets swapped.

```

class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:

```

```

if root is None:
    return None
stack = [root]
while stack:
    node = stack.pop()
    node.left, node.right = node.right, node.left
    if node.left: stack.append(node.left)
    if node.right: stack.append(node.right)
return root

```

Comparison with Reference Solutions

- `solution2.py` is essentially identical to my recommended solution — it's the cleanest, most idiomatic form. ✓
- `solution1.py` uses a helper `inverter` function. It works but adds unnecessary complexity; the recursion can swap directly without a separate helper.
- `my_solution.py` manually checks `if root.left`/`if root.right` before recursing. This is correct but verbose — the base case `if root is None` inside the recursive call already handles `None` children gracefully.

Recommendation: Use the recursive one-liner (Solution 2 style) for clarity, or the iterative BFS/DFS if you're worried about stack depth on very tall trees.

Codex Solution

1. Intuition

To **invert** a binary tree, every node's left and right children must be swapped.

For example:



becomes:



The key insight is:

If we swap the left and right child of every node, the whole tree becomes inverted.

This is naturally solved using **Depth-First Search recursion** because a binary tree is recursively defined:

- Invert the left subtree
- Invert the right subtree
- Swap them

Or equivalently:

- Swap current node's children
- Recursively invert both children

Both orders work.

2. Approach

We use recursive DFS.

Pseudocode

```

function invertTree(root):
    if root is null:
        return null

    invert left subtree
    invert right subtree

    swap root.left and root.right

```

```
return root
```

More compactly:

```
function invertTree(root):
    if root is null:
        return null

    root.left, root.right = invertTree(root.right), invertTree(root.left)

    return root
```

3. Python Solution

```
from typing import Optional

# Definition for a binary tree node.
# LeetCode already provides this class.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        # Base case:
        # If the current node is None, there is nothing to invert.
        if root is None:
            return None

        # Recursively invert the right subtree and make it the new left subtree.
        # Recursively invert the left subtree and make it the new right subtree.
        #
        # Python evaluates the right-hand side first, so this swap is safe.
        root.left, root.right = self.invertTree(root.right), self.invertTree(root.left)

        # Return the root of the inverted tree.
        return root
```

4. Complexity Analysis

Let n be the number of nodes in the tree.

Time Complexity: $O(n)$

We visit every node exactly once.

At each node, we perform a constant amount of work:

- Recursive calls
- Swapping left and right children

Therefore, total time complexity is:

```
 $O(n)$ 
```

Space Complexity: $O(h)$

The recursion stack depends on the height of the tree.

Where h is the height of the binary tree.

- For a balanced tree: $h = \log n$
- For a completely skewed tree: $h = n$

So the space complexity is:

```
 $O(h)$ 
```

Worst case:

$O(n)$

5. Alternative Approaches

Iterative DFS Using Stack

Instead of recursion, we can use a stack.

```
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if root is None:
            return None

        stack = [root]

        while stack:
            node = stack.pop()

            # Swap children
            node.left, node.right = node.right, node.left

            # Add children to stack if they exist
            if node.left:
                stack.append(node.left)
            if node.right:
                stack.append(node.right)

        return root
```

Time complexity: $O(n)$

Space complexity: $O(h)$ on average, $O(n)$ in the worst case.

Iterative BFS Using Queue

We can also traverse level by level using a queue.

```
from collections import deque

class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if root is None:
            return None

        queue = deque([root])

        while queue:
            node = queue.popleft()

            # Swap children
            node.left, node.right = node.right, node.left

            # Add children to queue
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        return root
```

Time complexity: $O(n)$

Space complexity: $O(n)$ in the worst case.

The recursive DFS solution is the cleanest and most common approach for this problem.